

Clustering jerárquico con scikit-learn sobre datos de tracking de jugadores de la NFL

Momento de Retroalimentación: Módulo 2 — Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución (Portafolio Implementación)

Karol Alexis Alvarado Davila — A01751711

Inteligencia artificial avanzada para la ciencia de datos

Grupo TC3009C.601

Tecnológico de Monterrey

Martes 8 de septiembre de 2026

1. Descripción del dataset

El dataset que usé es el mismo del proyecto anterior: tracking de jugadores de la NFL (archivo `input_2023_w01.csv`), que corresponde a la semana 1 de la temporada 2023. Tiene 285,714 registros y 23 columnas, sin valores nulos. Cada registro es un instante (frame) de un jugador dentro de una jugada, así que un mismo jugador aparece muchas veces: en total hay 16 partidos, 819 jugadas y 737 jugadores distintos, con unos 28 frames por jugador en cada jugada.

Las columnas se pueden agrupar así:

- Identificadores: `game_id`, `play_id`, `nfl_id`, `frame_id`.
- Datos del jugador: `player_name`, `player_height`, `player_weight`, `player_birth_date`, `player_position`, `player_side`, `player_role`.
- Contexto de la jugada: `play_direction`, `absolute_yardline_number`, `player_to_predict`, `num_frames_output`, `ball_land_x`, `ball_land_y`.
- Movimiento del jugador en ese instante: `x`, `y` (posición en el campo, en yardas), `s` (velocidad, yd/s), `a` (aceleración, yd/s²), `dir` (dirección del movimiento, en grados) y `o` (orientación del cuerpo, en grados).

2. Variables utilizadas y justificación

Para el clustering volví a usar solamente la velocidad `s` y la aceleración `a` (Cuadro 1). Las razones son las mismas que en el proyecto anterior, y se aplican igual porque el clustering jerárquico también agrupa por distancias: los ids no tienen una distancia con significado, `x` y `y` agruparían por zona del campo y no por tipo de movimiento, `dir` y `o` son ángulos donde 359° y 1° quedan lejos con la distancia euclidiana, y las columnas categóricas no se pueden meter a una distancia sin inventar números arbitrarios.

Variable	Media	Desv. est.	Mín.	Mediana	Máy.
<code>s</code> (yd/s)	3.04	2.23	0	2.73	12.53
<code>a</code> (yd/s ²)	2.13	1.43	0	1.92	17.12

Cuadro 1: Estadísticas de las variables utilizadas (285,714 registros).

Pensé en agregar una tercera variable como `player_weight`, pero la descarté por dos motivos. Primero, el peso es una característica fija del jugador, no del movimiento, y mezclarla con `s` y `a` haría que los clusters ya no representaran tipos de movimiento sino una combinación de tipo de jugador y movimiento que es más difícil de interpretar. Segundo, al usar exactamente las mismas dos variables y la misma muestra que en el proyecto anterior puedo comparar directamente el resultado del clustering jerárquico contra el de K-Means, que es lo que uso más adelante para la matriz de confusión.

3. Objetivo del proyecto

El objetivo es agrupar los registros de tracking según su velocidad y aceleración para encontrar tipos de movimiento parecidos, pero ahora usando el clustering jerárquico aglomerativo del framework `scikit-learn` (y `scipy` para el dendrograma), configurando bien sus parámetros: la métrica de distancia, el tipo de linkage y el número de clusters o punto de corte. Sigue siendo un problema de clustering, no de clasificación: no hay una etiqueta correcta que predecir.

4. Clustering jerárquico aglomerativo

El clustering jerárquico aglomerativo (bottom-up) funciona como vimos en clase:

1. Cada instancia empieza siendo su propio cluster.
2. Se calcula la distancia entre todos los clusters.
3. Se unen los dos clusters más cercanos en uno solo.
4. Se repite el paso 2 hasta que todo queda en un único cluster.

El resultado de todas esas uniones es el dendrograma, y el número de clusters se decide con un punto de corte: los clusters son las líneas verticales que cruza el corte. Para decidir qué tan “cerca” está un cluster de otro se usa una métrica de linkage. Las que vimos son single (par de puntos más cercano), complete (par más lejano), average (promedio de todos los pares), centroid (distancia entre centroides) y ward, que une los dos clusters cuya unión aumenta menos la varianza (la inercia) dentro de los clusters. La distancia entre puntos es la euclidiana.

Una diferencia importante con K-Means es que el algoritmo aglomerativo no tiene centroides ni una función `predict`: solo asigna cluster a los datos con los que se construyó el árbol. Por eso para los datos de prueba tuve que agregar un paso de asignación, que explico en la Sección 10.

5. Implementación con el framework

El programa está en `proyecto.py` y corre desde consola con `python proyecto.py`. El Cuadro 2 resume qué herramienta hace cada parte.

Parte del proyecto	Cómo se hizo
Leer el CSV, explorar y seleccionar s y a	pandas
Tomar la muestra (1 de cada 50 registros)	slicing de pandas
Estandarizar las variables	StandardScaler
Separar entrenamiento y prueba	train_test_split
Clustering jerárquico y comparación de linkages	AgglomerativeClustering de sklearn
Dendrograma	dendrogram de scipy
Elección de K	silhouette_score de sklearn
Asignación de los datos de prueba a un cluster	centroide más cercano con numpy
Inercia e índice de Dunn	funciones propias con numpy
Matriz de confusión contra K-Means	KMeans y confusion_matrix de sklearn
Gráficas	matplotlib

Cuadro 2: Herramientas usadas en cada parte del proyecto.

Estas son las partes principales del código donde se configura el algoritmo:

```

1  # comparacion de linkages con K = 3
2  tipos_linkage = ["ward", "complete", "average", "single"]
3  for tipo in tipos_linkage:
4      modelo = AgglomerativeClustering(n_clusters=3, metric="euclidean",
5                                       linkage=tipo)
6      etiquetas = modelo.fit_predict(X_train)
7      print(tipo, silhouette_score(X_train, etiquetas), np.bincount(
8            etiquetas))
9
10 # arbol completo para el dendrograma
11 modelo_completo = AgglomerativeClustering(distance_threshold=0,
12                                             n_clusters=None,
13                                             metric="euclidean", linkage="
14                                             ward")
15 modelo_completo = modelo_completo.fit(X_train)
16 plot_dendrogram(modelo_completo, truncate_mode="level", p=5)
17
18 # modelo final
19 modelo = AgglomerativeClustering(n_clusters=3, metric="euclidean",
20                                   linkage="ward")
21 etiquetas_train = modelo.fit_predict(X_train)

```

Los parámetros que usé y por qué:

- **metric="euclidean"**: la distancia entre puntos que vimos en clase y la única que acepta **ward**.
- **linkage="ward"**: elegido después de comparar contra **complete**, **average** y **single** (Sección 7).
- **n_clusters=3**: elegido con el dendrograma y el **silhouette** (Sección 8).
- **distance_threshold=0** con **n_clusters=None**: solo para el modelo del dendrograma, hace que se construya el árbol completo y se guarden las distancias de cada unión (**distances_**), que es lo que necesita **scipy** para dibujarlo. La función **plot_dendrogram** es la que vimos en clase.

Como el algoritmo no tiene `predict`, las funciones de apoyo que sí escribí yo son:

- `calcular_centroides(datos, etiquetas, k)`: promedio de los puntos de cada cluster.
- `asignar_clusters(datos, centroides)`: manda cada dato nuevo al centroide más cercano.
- `inercia` e `indice_dunn`: métricas de evaluación (Sección 11).

Ninguna de ellas es parte del algoritmo de clustering; son para evaluarlo y para usarlo con datos nuevos.

6. Datos de entrenamiento y datos de prueba

6.1. Muestra

El clustering jerárquico calcula la distancia entre todos los pares de puntos, así que su memoria crece con el cuadrado del número de registros; con los 285,714 registros originales no es posible correrlo. Por eso tomé la misma muestra del proyecto anterior: 1 registro de cada 50 usando slicing (`df[:50]`), que da 5,715 registros repartidos entre los 16 partidos del archivo. Con ese tamaño el algoritmo corre en segundos.

6.2. Escalamiento

Como el algoritmo usa distancias, estandaricé las dos variables con `StandardScaler` (media 0 y desviación estándar 1), para que ninguna pese más que la otra solo por tener valores más grandes.

6.3. Separación

Separé la muestra con `train_test_split` en 80 % para entrenamiento y 20 % para prueba, con `random_state=1` para que el resultado sea el mismo cada vez que se corre el programa (Cuadro 3).

Conjunto	Registros
Dataset original	285,714
Muestra (1 de cada 50)	5,715
Entrenamiento (80 %)	4,572
Prueba (20 %)	1,143

Cuadro 3: Datos utilizados para entrenar y para probar.

El conjunto de **entrenamiento** se usó para todo lo que es aprender: comparar linkages, construir el dendrograma, elegir K y obtener los clusters finales. El conjunto de **prueba** no se tocó en nada de eso; solo se usó al final para asignar datos nuevos a los clusters y calcular las métricas sobre datos que el algoritmo no había visto.

7. Elección del tipo de linkage

Corrí `AgglomerativeClustering` con $K = 3$ sobre el conjunto de entrenamiento cambiando solo el linkage, y comparé el silhouette y el tamaño de los clusters (Cuadro 4).

Linkage	Silhouette	Tamaño de los clusters
ward	0.3829	1236, 1486, 1850
complete	0.3405	3084, 264, 1224
average	0.3456	2742, 9, 1821
single	0.4614	4568, 1, 3

Cuadro 4: Comparación de linkages con $K = 3$ en el conjunto de entrenamiento.

Aquí se ve algo que también vimos en clase: el clustering jerárquico es sensible a los outliers. **single** tiene el silhouette más alto, pero es engañoso: sus “clusters” son un cluster con 4,568 puntos y dos con 1 y 3 puntos (los registros con aceleración extrema), así que en realidad no separa nada. **average** tiene el mismo problema con un cluster de 9 puntos. **complete** sí da tres grupos de tamaño razonable pero con un silhouette menor. **ward** da el mejor silhouette entre los que forman clusters de verdad y los tres grupos quedan balanceados, así que ese fue el que usé.

8. Número de clusters

Para escoger K usé las dos herramientas del tema: el dendrograma y el silhouette. La Figura 1 muestra los últimos 5 niveles del dendrograma con linkage ward, y el Cuadro 5 las distancias de las últimas uniones, que son las que definen dónde cortar.

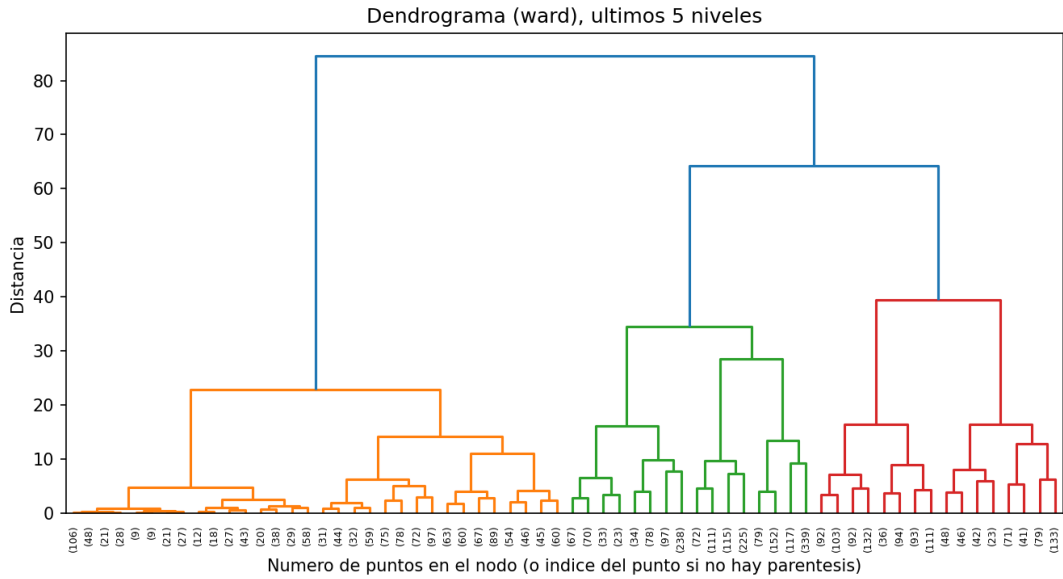


Figura 1: Dendrograma del conjunto de entrenamiento (ward), truncado a los últimos 5 niveles. Entre paréntesis está el número de puntos de cada rama.

Unión	Distancia	Clusters que quedan
1	16.45	8
2	16.47	7
3	22.77	6
4	28.47	5
5	34.54	4
6	39.49	3
7	64.29	2
8	84.52	1

Cuadro 5: Distancia de las últimas uniones del dendrograma (ward).

Las dos últimas uniones son las más caras: pasar de 3 a 2 clusters cuesta 64.3 y de 2 a 1 cuesta 84.5, mientras que de 4 a 3 solo cuesta 39.5 y hacia abajo las uniones son cada vez más baratas. Un corte a cualquier altura entre 40 y 64 cruza tres líneas verticales, es decir, define 3 clusters, y esa es la franja más ancha del dendrograma donde el número de clusters se mantiene estable.

Para confirmarlo calculé el silhouette con K de 2 a 8 (Cuadro 6 y Figura 2). El máximo está en $K = 3$, así que ese fue el valor que usé. Además coincide con el K del proyecto anterior, lo que permite comparar los dos métodos.

K	Silhouette
2	0.3478
3	0.3829
4	0.3628
5	0.3495
6	0.3521
7	0.3207
8	0.3123

Cuadro 6: Silhouette en el conjunto de entrenamiento para cada valor de K (ward).

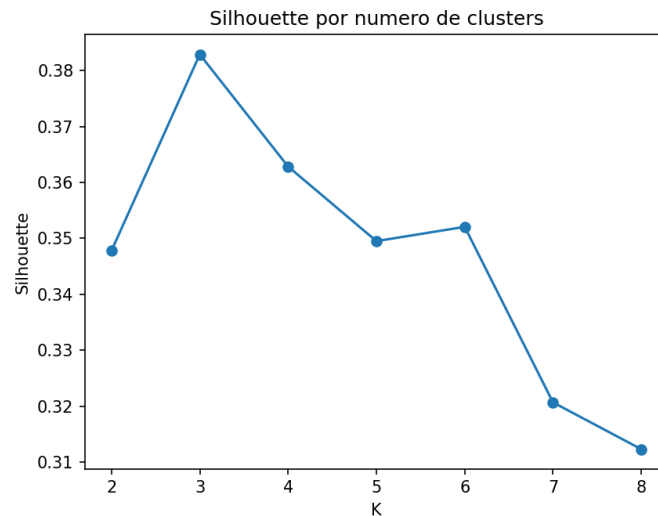


Figura 2: Silhouette contra número de clusters.

9. Resultados del entrenamiento

Con `n_clusters=3`, `linkage="ward"` y `metric="euclidean"` el modelo hizo 4,571 uniones (una menos que el número de puntos, como debe ser) y dejó tres clusters. El Cuadro 7 muestra el centroide de cada uno, calculado como el promedio de sus puntos: primero en las unidades escaladas y después en unidades originales, que es lo que permite interpretarlos.

Cluster	Centroide escalado (s, a)	Registros	$\bar{s}$ (yd/s)	$\bar{a}$ (yd/s ²)	Interpretación
0	(-0,247, 1,138)	1236	2.51	3.71	Acelerando
1	(-0,946, -0,862)	1486	0.94	0.88	Detenido o caminando
2	(0,906, -0,070)	1850	5.11	2.00	Corriendo rápido y estable

Cuadro 7: Centroides y descripción de cada cluster en el conjunto de entrenamiento.

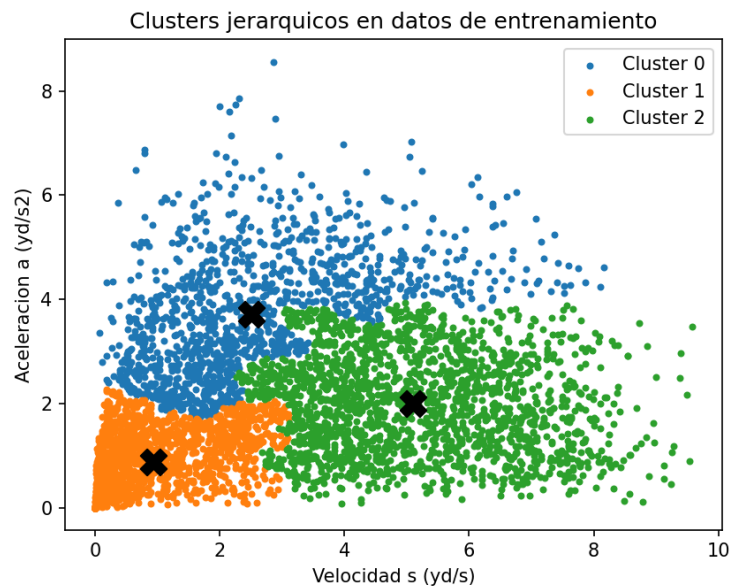


Figura 3: Clusters jerárquicos en el conjunto de entrenamiento. Cada color es un cluster y la X negra es su centroide.

En la Figura 3 se ve que los tres grupos ocupan las mismas zonas del plano velocidad–aceleración que encontró K-Means: uno en la esquina de valores bajos, uno a la derecha (velocidad alta) y uno arriba (aceleración alta). Lo que cambia es dónde quedan las fronteras: el cluster de velocidad alta es más grande aquí (1,850 registros contra 1,327) porque ward le asignó una parte de los registros con velocidad media que K-Means dejaba en el cluster de detenidos. Los números de cluster también son distintos (aquí el 1 es el de detenidos), porque cada algoritmo numera sus grupos en el orden en que los encuentra.

10. Asignación de los datos de prueba

Como `AgglomerativeClustering` no tiene `predict`, para usar el modelo con datos nuevos calculé el centroide de cada cluster del entrenamiento y asigné cada registro de prueba al centroide más cercano con distancia euclidiana (función `asignar_clusters`). Es la forma natural de extender un cluster a datos nuevos: un registro pertenece al grupo cuyo “punto típico” le queda más cerca, y el árbol no se recalcula. El Cuadro 8 muestra los primeros 10 registros de prueba con el cluster que les tocó.

s (yd/s)	a (yd/s ²)	Rol del jugador	Cluster asignado
0.65	2.60	Defensive Coverage	0
3.55	0.37	Defensive Coverage	1
6.78	3.64	Other Route Runner	2
2.45	2.33	Passer	0
1.78	3.10	Defensive Coverage	0
0.18	1.80	Passer	1
1.45	1.53	Defensive Coverage	1
6.20	2.22	Defensive Coverage	2
1.75	1.49	Defensive Coverage	1
0.89	2.13	Defensive Coverage	1

Cuadro 8: Ejemplos de registros de prueba y el cluster asignado.

Las asignaciones tienen sentido: los registros con velocidad alta (6.78 y 6.20) quedan en el cluster 2, los de aceleración alta (2.60, 2.33, 3.10) en el cluster 0 y los casi detenidos en el cluster 1. El Cuadro 9 compara el reparto entre clusters en entrenamiento y en prueba, y la Figura 4 muestra los datos de prueba con su cluster.

Cluster	Entrenamiento	%	Prueba	%
0	1236	27.0	334	29.2
1	1486	32.5	407	35.6
2	1850	40.5	402	35.2
Total	4572	100	1143	100

Cuadro 9: Registros por cluster en entrenamiento y en prueba.

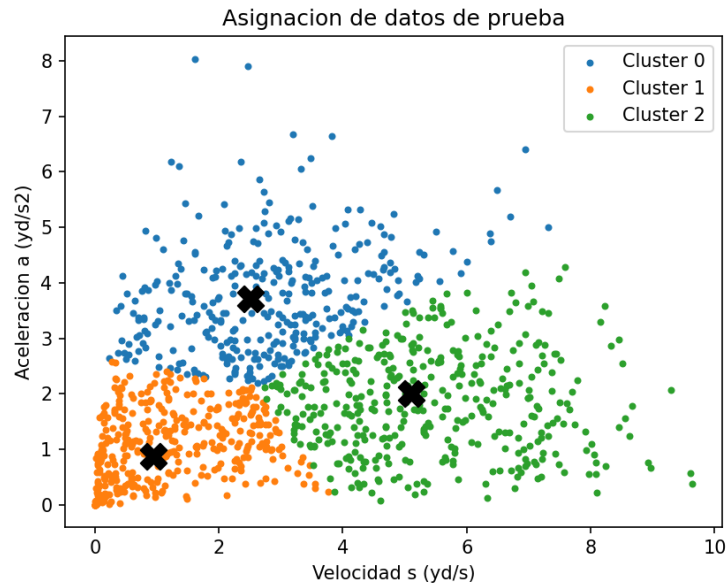


Figura 4: Datos de prueba asignados a los clusters. Las X negras son los centroides obtenidos en el entrenamiento.

11. Métricas y matriz de confusión

11.1. Métricas seleccionadas

Usé tres métricas de clustering, dos del tema y una más del framework:

- **Silhouette** (`silhouette_score` de `sklearn`): compara qué tan cerca está cada punto de su propio cluster contra qué tan cerca está del cluster vecino más próximo. Va de -1 a 1 ; entre más alto, mejor definidos los clusters.
- **Inercia** (distancia intracluster): suma de las distancias al cuadrado de cada punto a su centroide. Entre más chica, más compactos son los clusters. La divido entre el número de puntos para poder comparar entrenamiento y prueba.
- **Índice de Dunn** (distancia intercluster): mínima distancia entre centroides dividida entre la máxima distancia promedio de los puntos a su centroide. Entre más grande, más separados están los clusters.

Las tres se calcularon con los mismos centroides del entrenamiento (Cuadro 10).

Métrica	Entrenamiento	Prueba
Registros	4572	1143
Silhouette	0.3829	0.4205
Inercia total	3591.99	801.03
Inercia promedio por punto	0.7856	0.7008
Índice de Dunn	1.7520	1.9810

Cuadro 10: Métricas de evaluación en entrenamiento y en prueba.

Las métricas en prueba son iguales o un poco mejores que en entrenamiento: la inercia por punto baja de 0.79 a 0.70, y el silhouette y el Dunn suben. Esto quiere decir que los centroides que salieron del árbol representan bien los tipos de movimiento en general y no solo a la muestra de entrenamiento. Que las métricas de prueba sean ligeramente mejores tiene una explicación: los puntos de prueba se asignaron al centroide más cercano, que es la asignación que minimiza la inercia por construcción, mientras que en entrenamiento las etiquetas las decidió el árbol de ward, que junta grupos completos y puede dejar algún punto más cerca de otro centroide que del suyo.

11.2. Matriz de confusión

Este proyecto no tiene una etiqueta verdadera contra la cual medir aciertos: no existe una columna que diga si un registro es “detenido”, “acelerando” o “corriendo”, y las categorías del dataset (`player_role`, `player_position`) no describen movimiento, porque un mismo receptor puede estar parado en un frame y a máxima velocidad en otro. Por eso una matriz de confusión contra esas columnas no mediría nada útil.

Lo que sí es una comparación válida es confrontar los clusters del jerárquico contra los de otro algoritmo sobre exactamente los mismos datos. Entrené `KMeans` de `sklearn` con $K = 3$ sobre el mismo conjunto de entrenamiento y usé `confusion_matrix` de `sklearn` para cruzar las dos etiquetas. Como cada método numera sus clusters en un orden distinto, primero emparejé cada cluster jerárquico con el cluster de K-Means con el que comparte más registros. El Cuadro 11 muestra el resultado en entrenamiento y en prueba (en prueba, las etiquetas de K-Means vienen de su `predict` y las del jerárquico de `asignar_clusters`).

Jerárquico	K-Means (entrenamiento)			K-Means (prueba)		
	Acelerando	Detenido	Corriendo	Acelerando	Detenido	Corriendo
Acelerando	935	293	8	299	35	0
Detenido	0	1486	0	0	407	0
Corriendo	361	172	1317	33	31	338

Cuadro 11: Matriz de confusión entre los clusters del jerárquico (filas) y los de K-Means (columnas). La diagonal son los registros en los que los dos métodos coinciden.

La concordancia (suma de la diagonal entre el total) es de 81.8 % en entrenamiento y 91.3 % en prueba. Leyendo la matriz:

- El cluster de *detenidos* es idéntico en los dos métodos: los 1,486 registros que ward puso ahí también los puso K-Means, y en prueba pasa lo mismo con los 407.
- Los desacuerdos están en las fronteras entre *acelerando* y *corriendo* (361 registros) y entre *acelerando* y *detenido* (293). Son los registros con velocidad y aceleración medias, que están en la zona continua entre los grupos y que cualquiera de los dos métodos podría poner de un lado o del otro.
- La concordancia sube en prueba porque ahí los dos métodos asignan por centroide más cercano, y la única diferencia que queda es la posición de los centroides.

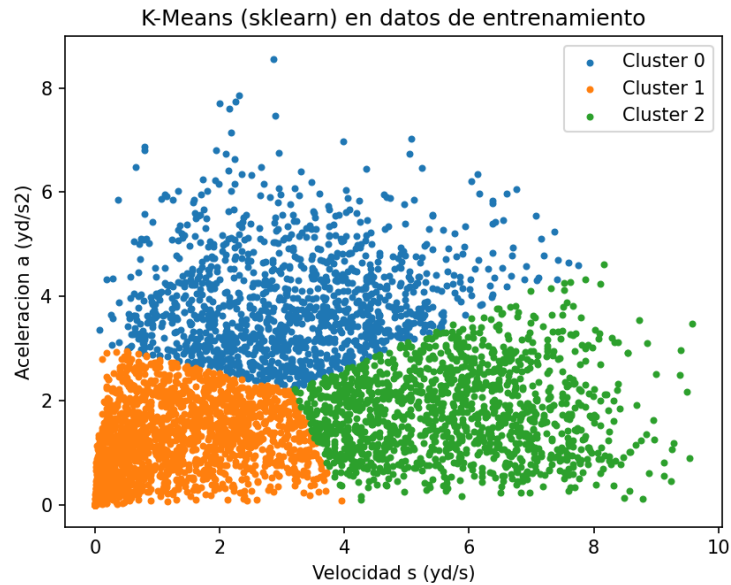


Figura 5: Los mismos datos de entrenamiento agrupados por K-Means de `sklearn`, con los números de cluster ya emparejados con los del jerárquico. Compárese con la Figura 3.

Es importante decir qué mide y qué no mide esta matriz: mide qué tanto coinciden dos algoritmos distintos, no cuál de los dos está “bien”, porque ninguno tiene la respuesta correcta. Que dos métodos que funcionan de forma muy distinta (uno mueve centroides, el otro une vecinos de abajo hacia arriba) lleguen a los mismos tres grupos con un acuerdo de más del 80 % es evidencia de que los tres tipos de movimiento están de verdad en los datos y no son un artefacto de un algoritmo en particular.

12. Análisis de resultados

Los resultados muestran que el framework quedó bien configurado y que el algoritmo hace lo que se espera:

- La comparación de linkages muestra por qué la elección de parámetros importa: con `single` y `average` el algoritmo se deja llevar por los outliers y forma clusters de 1, 3 o 9 puntos; con `ward` los tres grupos quedan balanceados y con el mejor silhouette entre los linkages que sí separan algo.
- El dendrograma tiene un salto claro: la unión de 3 a 2 clusters cuesta 64.3 contra 39.5 de la anterior, y el silhouette confirma el máximo en $K = 3$. Dos criterios independientes llevan al mismo K .
- Los tres clusters se interpretan sin forzar nada: detenido o caminando, acelerando y corriendo rápido y estable, igual que en el proyecto anterior.
- Los datos de prueba se asignan con métricas iguales o mejores que las de entrenamiento, y la matriz de confusión contra K-Means da una concordancia de 82 % en entrenamiento y 91 % en prueba, con el cluster de detenidos idéntico entre métodos.

También hay que tener en cuenta las limitaciones. El silhouette de 0.38 no es alto: como ya vimos con K-Means, la nube de velocidad–aceleración es continua y no tiene fronteras naturales muy marcadas, así que cualquier partición en tres deja puntos ambiguos en los bordes; la matriz de confusión los ubica justo ahí. El clustering jerárquico además es más costoso que K-Means (memoria cuadrática), por eso trabajé con la muestra de 5,715 registros y no con el dataset completo. Y como el algoritmo no tiene `predict`, la asignación de datos nuevos por centroide más cercano es una aproximación: es la extensión más razonable, pero un punto nuevo podría haber quedado en otra rama si hubiera estado en el árbol desde el principio.

13. Conclusión

Implementé clustering jerárquico aglomerativo con `AgglomerativeClustering` de `scikit-learn` y `scipy` para el dendrograma, sobre 4,572 registros de entrenamiento y 1,143 de prueba del dataset de tracking de la NFL. Configuré el algoritmo comparando cuatro tipos de linkage, construyendo el árbol completo con `distance_threshold=0` para leer el dendrograma, y eligiendo $K = 3$ con el punto de corte y el silhouette. Como el algoritmo no predice datos nuevos, agregué la asignación por centroide más cercano.

Con `linkage="ward"` y $K = 3$ obtuve los mismos tres tipos de movimiento que K-Means, con silhouette de 0.42, inercia por punto de 0.70 e índice de Dunn de 1.98 en prueba, y una concordancia de 91 % con K-Means en los datos de prueba. Con esto queda demostrado que sé configurar el algoritmo del framework, que el modelo generaliza a datos que no había visto y que los grupos encontrados son consistentes entre dos métodos distintos.